

DevMultiGroup — AI Agents Bootcamp

Sunum İçeriğinin Detaylı Özeti

Bu doküman, "DevMultiGroup - AI Agents BootCamp" sunumunun tüm bölümlerinin detaylı içerik özetidir. Sunumdaki altı ana modül, modüllerin öğretim akışı, temel kavramlar, karşılaştırma tabloları ve kapanış mesajları bölüm bölüm işlenmiştir.

1. Sunumun Genel Yapısı ve Anlatı Mantığı

Sunum, tek bir büyük soruyu adım adım cevaplayacak biçimde kurgulanmıştır: "Yapay zekâ nedir ve bir yapay zekâ ajanı gerçek dünyada nasıl güvenle çalışır?" Bu nedenle modüller birbirinin üzerine inşa edilir; her modül sonraki modülün ön koşulu olan kavramı verir.

#	Modül	Öğrettiği temel kavram	Sonraki modüle bağlantısı
1	Yapay Zekâ Nedir?	AI / ML / DL ilişkisi, veri kalitesi, model-algoritma ayrımı, halüsinasyon	Derin öğrenmenin neden gerekli olduğunu kurar
2	Yapay Zekânın Tarihi	Kurallardan öğrenmeye, öğrenmeden temel modellere 80 yıllık evrim	Transformer ve LLM dönemine zemin hazırlar
3	Derin Öğrenme ve LLM'lere Geçiş	Sinir ağları, parametre-veri-maliyet üçgeni, sonraki kelime tahmini	LLM mekaniğinin giriş kapısı
4	LLM'ler Nasıl Çalışır?	Token, embedding, attention, inference, RAG, güvenlik, değerlendirme	Ajan mimarisinin "beyin" katmanını tanımlar
5	Agentic AI ve Ajanlar	Ajan döngüsü, araçlar, otonomi seviyeleri, guardrail, orkestrasyon	Ajanların araçlara nasıl bağlandığı sorusunu doğurur
6	MCP (Model Context Protocol)	Standart araç/bağlam katmanı, host-client-server, M+N ölçeği	Uygulamaya geçiş: "bu hafta bir tane bağla, bir tane yaz"

3. Modül 1 — Yapay Zekâ Nedir?

3.1 Günlük hayatta yapay zekâ

Ana tez: AI çoğu zaman görünmezdir — önerir, sıralar, tahmin eder, filtreler ve uyarır. Farkında olmadan kullanılır.

Uygulama	AI'nın yaptığı iş
Spotify	Kişisel müzik önerileri
Instagram	İçerik sıralama
Bankacılık	Şüpheli işlem tespiti
E-posta	Spam filtreleme
Navigasyon	Trafik tahmini

3.2 AI, Makine Öğrenmesi ve Derin Öğrenme ilişkisi

- AI:** insan zekâsı gerektiren görevleri gerçekleştirmeyi amaçlayan geniş alan (uzman sistemler, robotik, konuşma teknolojileri, otonom sistemler dahil).
- ML:** sistemlerin açık kurallar yerine veriden öğrenmesini sağlayan yaklaşım — gözetimli, gözetimsiz ve pekiştirmeli öğrenme.
- DL:** çok katmanlı yapay sinir ağları kullanan makine öğrenmesi yaklaşımı; üretken AI'nın temeli.

Kritik ayırım: Her ML sistemi bir AI uygulaması olabilir; ancak her AI sistemi ML kullanmak zorunda değildir.

3.3 Yapay zekâ kategorileri

Kategori	Açılım	Anlamı / örnek
ANI	Artificial Narrow Intelligence	Dar alanda uzmanlaşmış zekâ — akıllı hoparlör, web arama
AGI	Artificial General Intelligence	İnsanın yapabildiği her şeyi yapabilen genel zekâ
GenAI	Generative AI	Yeni içerik üreten sistemler — ChatGPT, Bard, DALL·E

3.4 Sistem davranışına göre sınıflandırma

AI sistemleri ürettikleri davranış biçimine göre üç temel grupta ele alınır.

Tür	Girdi	Çıktı	Temel soru	Örnek
01 Reaktif	Kural / koşul	Tepki	Ne yapmalıyım?	Belirli kelimeyi engelleme, basit oyun kuralları
02 Tahminsel	Geçmiş veri	Skor / tahmin	Ne olabilir?	Satın alma ihtimali, hasta risk skoru
03 Üretken	İstem / bağlam	Yeni içerik	Ne üretebilirim?	Video senaryosu, müşteri e-postası taslağı

Not: Aynı iş akışında bu üç davranış biçimi birlikte kullanılabilir; amaçları sırasıyla kuralı uygulamak, olasılığı hesaplamak ve yeni çıktı üretmektir.

3.5 Veri kalitesi: Garbage In, Garbage Out

Model, öğrendiği verinin doğruluğu ve temsil gücü kadar güvenilirdir. Kalitesiz veri → AI modeli → güvenilirmez sonuç zinciri sunumun merkezî uyarısıdır.

Kalitesiz verinin belirtileri ve sonuçları

- Belirtiler: eksik, hatalı, dengesiz, yanlış etiketli veri.
- Sonuçlar: yanlış tahmin, taraflı karar, güven kaybı.

İyi bir veri sisteminin beş özelliği

- **Doğru** — hatalar ve tutarsızlıklar azaltılmış.
- **Güncel** — mevcut durumu yansıtıyor.
- **Tutarlı** — aynı standartlarla toplanmış.
- **Güvenilir etiketli** — hedef sonuç doğru tanımlanmış.
- **Yasal ve etik** — uygun izinlerle erişilmiş.

3.6 Algoritma ve model ayrımı

Algoritma öğrenme yöntemidir; model, bu yöntemin verilerle eğitilmesi sonucu oluşan öğrenilmiş yapıdır. Aynı algoritma farklı verilerle farklı model üretir.

Mutfak metaforu	Teknik karşılığı
Yemek tarifi	Algoritma (Linear Regression, Decision Tree, K-Means)
Malzemeler	Eğitim verisi
Hazırlama	Eğitim süreci
Ortaya çıkan yemek	Model
Yeni girdiye üretilen sonuç	Tahmin

Ana mesaj: Tarif aynı kalsa da malzemeler değiştiğinde ortaya çıkan yemek — yani model — değişir.

3.7 Sınırlılıklar, hata kaynakları ve halüsinasyon

AI güçlü örüntüler bulabilir; ancak bağlamı eksik yorumlayabilir, önyargı taşıyabilir ve sonucunu açıklamakta zorlanabilir.

Halüsinasyon nedir?

Gerçekte olmayan veya doğrulanmamış bilginin ikna edici biçimde üretilmesidir. Tipik biçimleri:

- Olmayan kaynak, yasa veya ürün özelliği uydurma.
- Yanlış tarih, alıntı ya da sayısal bilgi verme.
- Hatalı kodu doğru çözüm gibi sunma.

Neden olur?

Dil modelleri öncelikle doğruyu kanıtlamaya değil, en olası kelime dizisini üretmeye çalışır. Bu nedenle ikna edici görünüm, doğrulanmış bilgi anlamına gelmez.

Güvenli kullanım zinciri

- AI çıktısı akıcı ve güvenli görünebilir — bu bir doğruluk kanıtı değildir.
- Doğrulama: kaynak, tarih ve kritik iddiaları kontrol et.
- Güvenli kullanım: insan onayı, erişim sınırı ve güvenilir bilgi tabanı.

Uyarı: Özellikle hukuk, sađlık, finans ve müşteri iletişiminde sonuçlar kontrol edilmeden kullanılmamalıdır.

3.8 Başarılı bir AI projesinin bileşenleri

Başarı yalnızca iyi bir model kurmak değildir. İş hedefi, veri, insan, süreç ve teknoloji birlikte çalıştığında AI gerçek değer üretir.

Formül: Teknoloji + veri + süreç + insan = sürdürülebilir etki.

4. Modül 2 — Yapay Zekânın Tarihi

Alt başlık: "Seksen Yıllık Bilimsel Birikim ve Teknolojik Evrim". Bu modülün amacı, bugünkü sistemleri bir anda ortaya çıkmış bir sıçrama değil, birikimli bir evrim olarak göstermektir.

4.1 Modern öncesi temeller ve Alan Turing

- **1943 — McCulloch-Pitts modeli:** biyolojik sinir hücrelerinden esinlenen ilk yapay nöron modeli; modern derin öğrenmenin en temel yapı taşı. Girdi/çıktı mantığı: belirli bir eşik aşıldığında sinyal üretimi.
- **Üç temel gereksinim:** soyut mantık, matematiksel makineler ve programlanabilirlik.
- **Alan Turing (1912–1954):** zekâyı doğrudan tanımlamak yerine davranışsal performans üzerinden değerlendirme fikrini önerdi. "Makineler düşünebilir mi?" sorusu, Turing Testi ve hesaplanabilirlik tartışması. Amaç bilinç kanıtı değil, iletişim başarısıdır.

4.2 İlk uygulamalar: ELIZA ve uzman sistemler

ELIZA (1966)

- Geliştirici: Joseph Weizenbaum (MIT). Yöntem: kalıp eşleme ve kullanıcı ifadelerini soruya dönüştürme (DOCTOR senaryosu).
- **ELIZA Etkisi:** kullanıcıların, rasyonel bir temeli olmasa dahi bilgisayar programlarına insani duygu ve niyet atfetme eğilimi. Gerçek bir anlama yetisi yoktur; psikolojik bağ illüzyonu yaratır ve modern chatbot beklentilerinin temelidir.

Uzman sistemler

- Yaklaşım: insan uzmanların bilgisini If-Then kuralları biçiminde sisteme aktarmak.
- Öncü örnekler: DENDRAL (kimyasal moleküler yapı analizi), MYCIN (kan enfeksiyonları ve antibiyotik önerisi).
- Mimari bileşenler: bilgi tabanı, çıkarım mekanizması, kurallar.
- Kullanım alanları: arıza tespiti, kredi değerlendirme, tıbbi karar desteği, finansal analiz.

Sınırlılık: Bu sistemler kendi kendine öğrenemez; bilgi tabanının insanlar tarafından manuel güncellenmesi gerekir ve tanımlanmamış durumlarda kırılgandır.

4.3 Yapay zekâ kışları ve teknik sınırlar

AI kış döngüsü, yatırım ve beklentilerin azaldığı dönemlerin arkasındaki tekrarlayan süreçtir:

- Aşırı iyimserlik ve büyük beklentiler.
- Yüksek kamu ve özel sektör yatırımları.
- Verilen sözlerin yerine getirilememesi.
- Finansal desteğin ve ilginin kaybolması.

Birinci kış (1970'ler): erken vaatlerin teknik engellere çarpması. İkinci kış (1980'ler sonu): uzman sistemlerin sürdürülebilirlik sorunları.

Ana ders: Demo başarısı, gerçek dünya başarısını garanti etmez.

4.4 Veri odaklı dönüşüm ve makine öğrenmesi

Strateji, "uzman kurallarını sisteme aktarma" yönteminden "sisteme yeterli örnek gösterme" ilkesine evrildi: istatistiksel modeller, olasılıklar ve geçmiş veriden kalıp çıkarma. Spam filtreleme ve öneri sistemleri bu dönemde doğdu.

- **1997 — Deep Blue:** IBM Deep Blue, dünya satranç şampiyonu Garry Kasparov'u mağlup etti. Ders: belirli bir görevde insanı yenmek genel zekâya sahip olmak anlamına gelmez; Deep Blue satranca özelleşmişti.
- **Yeni yakıt — dijitalleşme:** web siteleri, e-ticaret ve sosyal medya verileri; ölçeklenen veritabanları ve sensör ağları.
- **GPU devrimi:** grafik işlem birimleri sinir ağı eğitiminde kritik avantaj sağladı.

Sonuç: Algoritma + veri + donanım birleşimi derin öğrenme devrimini tetikledi.

4.5 Derin öğrenme kırılmaları

- **2012 — AlexNet ve ImageNet:** derin öğrenmenin geleneksel yöntemleri geride bıraktığı ilk büyük dönüm noktası. Temsil ettiği soru: "Bu resimde ne var?"
- **2016 — AlphaGo:** makinelerin yalnızca insanı taklit etmeyip yeni stratejiler geliştirebileceğinin kanıtı. Temsil ettiği soru: "Şimdi hangi hamleyi yapmalıyım?"

4.6 Dil işlemede devrim: Transformer ve Attention

- Geleneksel sınırlamalar: RNN ve LSTM gibi sıralı modellerdeki temel engeller.
- Attention mekanizması: modelin girdi içindeki en alakalı bölümlere seçici odaklanması — örneğin bir zaminin cümlede hangi nesneye referans verdiğini anlamak için uzak kelimeler arasında bağ kurması.
- 2017 — "Attention Is All You Need" makalesiyle Transformer mimarisi.
- Yeni öğrenme paradigması: ön eğitim (pre-training) sonrası modellerin çok yönlü yetenek kazanması.

4.7 ChatGPT ve üretken yapay zekâ dönemi

30 Kasım 2022 tarihsel kırılma noktası olarak işlenir: ChatGPT yapay zekâyı icat etmedi; seksen yıllık bilimsel birikimi milyonlarca insanın doğrudan deneyimleyebileceği bir arayüze dönüştürdü.

- Diyalog bazlı doğal iletişim ve takip soruları.
- Hatalı öncülleri sorgulama yeteneği.
- Düşük başlangıç engeli ve hızlı geri bildirim.
- **Yeni kavram — Prompt:** doğal dilin bir kullanıcı arayüzüne dönüşmesi. Yazılım kullanmak yerine yazılımla konuşma dönemi başladı.

Kullanım alanları: içerik üretimi, kod yazımı, metin özetleme, çeviri ve analiz, fikir üretme, veri işleme.

4.8 Tarihsel zaman çizelgesi

Yıl	Kilometre taşı
1943	McCulloch-Pitts yapay nöron modeli
1950	Alan Turing: Turing Testi önerisi
1956	Dartmouth Konferansı: "Yapay Zekâ" teriminin doğuşu
1958	Frank Rosenblatt: Perceptron modeli
1966	ELIZA: ilk diyalog programı
1970–80	Yapay zekâ kışları ve uzman sistemler
1997	Deep Blue: Kasparov karşısında galibiyet
2011	IBM Watson: Jeopardy! başarısı

Yıl	Kilometre taşı
2012	AlexNet: derin öğrenme devrimi
2016	AlphaGo: Go oyununda tarihsel kırılma
2017	Transformer mimarisi ve dikkat mekanizması
2022	ChatGPT: üretken yapay zekânın yaygınlaşması

4.9 Beş büyük tarihsel ders

- **Demo ≠ Ürün:** kontrollü laboratuvar başarısı, gerçek dünya karmaşıklığında aynı sonucu garanti etmez.
- **Bütünsel başarı:** ilerleme sadece algoritmaya değil; veri, donanım, yazılım ve insan faktörünün birleşimine bağlıdır.
- **Beklenti yönetimi:** aşırı iyimser vaatler ve teknik sınırların göz ardı edilmesi "AI kışlarına" yol açar.
- **Dar odak avantajı:** genel zekâ yerine belirli ve dar bir problem alanına odaklanmak daha kontrollü başarı getirir.
- **Fikirlerin rönesansı:** geçmişte altyapı yetersizliğiyle başarısız olan fikirler yeni koşullarda (örn. GPU) devrim yapabilir.

4.10 Karşılaştırmalı analiz: yaklaşımların evrimi

Boyut	Sembolik YZ	Makine Öğrenmesi	Büyük Dil Modelleri
Bilgi kaynağı	İnsan uzmanlar — yapılandırılmış uzman bilgisi ve mantık kuralları	Geçmiş veriler — etiketli veri setleri ve istatistiksel örnekler	Devasa metin külliyatı — internet ölçeğinde ham ve çok modlu veri
Temel yöntem	Kurallar ve mantık (If-Then, arama, sembolik çıkarım)	İstatistiksel öğrenme — veriden kalıp çıkarma	Transformer & attention — ön eğitim ve bağlamsal ilişki
Öğrenme süreci	Sınırlı / manuel — bilgi tabanı insan tarafından güncellenir	Denetimli öğrenme — örneklerden genelleme	Ön eğitim & uyarılma — self-supervised + insan geri bildirimi
Açıklanabilirlik	Yüksek (şeffaf) — karar adımları izlenebilir	Değişken — modele göre (karar ağacı vs. derin sinir ağı)	Sınırlı (kara kutu) — milyarlarca parametre arası ilişki
Esneklik	Düşük — dar uzmanlık alanına bağlı	Orta — belirli görev için optimize	Çok yüksek — çok sayıda farklı görev
Örnek	MYCIN, DENDRAL	Spam filtresi, AlexNet	ChatGPT, Claude

Modül kapanışı: Yapay zekâ bugünün değil, seksen yıllık bilimsel birikimin ürünüdür. Tarih; kurallardan öğrenmeye, öğrenmeden büyük temel modellere uzanan bir yolculuktur. Sözlük: Turing Testi, Sembolik YZ, Perceptron, Uzman Sistemler, YZ Kışı, Derin Öğrenme, Transformer, Üretken YZ.

5. Modül 3 — Derin Öğrenme ve LLM'lere Geçiş

5.1 Derin öğrenme

- Derin öğrenme, büyük sinir ağlarının yeni adıdır; beyinden ve nöronların çalışma biçiminden esinlenmiştir.
- Yapay nöronları katmanlar halinde istiflemeye ve verileri işlemeleri için onları "eğitmeye" odaklanır. "Derin" sıfatı, ağda çok sayıda (yüzlerce veya binlerce) katman bulunmasını ifade eder.
- Gözetimli, gözetimsiz veya pekiştirmeli olarak kullanılabilir.
- Büyük dil modellerinin (LLM) ve üretken yapay zekânın (GenAI) temelidir.

Performans üçgeni: Bir derin öğrenme ağının performansı parametre sayısı, veri boyutu ve eğitim maliyetine bağlıdır.

5.2 Sinir ağı sezgisi: iki örnek

- **Emlak fiyatı örneği:** giriş parametreleri (konum, emlak türü, yatak odası sayısı, banyo sayısı, m²) → büyük matematiksel denklem (yapay sinir ağı) → çıkış (fiyat).
- **Sipariş tahmini örneği:** fiyat, teslimat maliyeti, pazarlama ve müşteri deneyimi girdileri; ara nöronlar satın alınabilirlik, farkındalık, kalite ve talep gibi soyut kavramları temsil eder; çıktı tahmin edilen sipariş sayısıdır.

5.3 Büyük dil modelleri (LLM) için gerekenler

- Muazzam miktarda veri.
- Güçlü sinir ağı makine öğrenimi algoritması.
- Süper bilgisayarlar.

LLM, sürekli olarak bir sonraki kelimeyi tahmin eden bir derin öğrenme algoritmasıdır; oto-regresif (AR) olarak da adlandırılır ve üretken yapay zekâ teknolojisinin temelidir.

Olasılık zinciri: $P(\text{metın}) = P(1. \text{ kelime}) \times P(2. \text{ kelime} \mid 1. \text{ kelime}) \times P(3. \text{ kelime} \mid 2. \text{ ve } 1. \text{ kelime}) \times \dots$ — yani her adımda bir sonraki kelimenin olasılığı.

6. Modül 4 — Büyük Dil Modelleri Nasıl Çalışır?

Bu modül, tokenlardan anlamlı çıktılara uzanan süreçte LLM'lerin çalışma mekanizmalarını, mimarisini ve kurumsal uygulama prensiplerini kapsar.

LLM çalışma zinciri: Metin → token → embedding → attention/Transformer katmanları → inference (token üretimi) → RAG ve araçlarla güncel/kurumsal bilgiye kontrollü erişim → güvenlik ve doğrulama (talimatlar, erişim sınırları, çıktı kontrolleri) → güvenilir yanıt.

6.1 LLM nedir, neler yapabilir?

LLM; büyük ölçekli eğitim verisi, yüksek hesaplama kapasitesi ve çok sayıda öğrenilmiş parametreyle metni işleyen ve üreten bir dil modelidir. Temel yetenekleri:

Yetenek	Ne yapar
Metin üretimi	Taslak, açıklama ve diyalog oluşturma
Soru yanıtlama	Bağlama göre yanıt üretme
Özetleme	Uzun içeriği ana fikirlere indirgeme
Çeviri	Diller arasında metin dönüştürme
Kod üretimi	Kod taslağı ve açıklama hazırlama
Belge analizi	Sınıflandırma ve bilgi çıkarımı

Uyarı: Güçlü yetenek ≠ doğruluk garantisi. Akıcı çıktı, doğru veya güncel bilgi anlamına gelmez.

6.2 LLM ile chatbot ayrımı

Model metni üretir; chatbot, bu modeli güvenli ve görev odaklı bir uygulamaya dönüştürür.

Formül: Chatbot = LLM + uygulama katmanları + veri + güvenlik.

6.3 Token ve tokenizasyon

Tokenizasyon, metnin modelin işleyebileceği anlamlı parçalara bölünmesi sürecidir; her token sayısal bir kimlik değeriyle temsil edilir. Örnek: "Bugün modeller öğreniyor." → Bugün / model / ler / öğren / iyor / . (kimlik değerleri modele ve tokenizer'a göre değişir).

- Kelime:** bazı sık kullanılan ifadeler tek token olabilir.
- Kelime parçası / ek:** özellikle eklemeli dillerde bir kelime birden fazla tokena ayrılabilir ("Kitaplarımızdan" → Kitap / lar / ımız / dan).
- Sayı, noktalama ve özel işaret:** boşluk, nokta, sayı veya kontrol işaretleri de token olabilir.

Tokenizasyon neden önemlidir?

Etki alanı	Neden önemli
Dil performansı	Kelime ve ek biçimlerinin doğru işlenmesini etkiler
Bağlam penceresi	Daha fazla token, kullanılabilir bağlamı daha hızlı tüketir
Maliyet	Girdi ve çıktı tokenları kullanım maliyetini belirler
Üretim hızı	İşlenecek ve üretilecek token sayısı gecikmeyi etkiler

Türkçe için kritik: Aynı anlam, farklı token sayısı ile farklı maliyet ve bağlam yükü oluşturabilir; eklemeli dillerde bu etki daha belirgindir.

6.4 Embedding, anlamsal benzerlik ve bağlam

- Token kimlikleri, modelin ilişkileri hesaplayabilmesi için çok boyutlu sayısal vektörlere dönüştürülür.
- Anlam tek bir sayıda değil, vektördeki birlikte konumda temsil edilir; görsel şemalar yalnızca sezgisel izdüşümdür.
- Anlamca benzer ifadeler vektör uzayında birbirine daha yakın konumlanır.
- Aynı kelime, bulunduğu cümleye göre farklı bağlamsal temsiller kazanabilir — yakınlık yalnızca kelimeye değil, kullanıldığı bağlama da bağlıdır.

6.5 Konum bilgisi ve context

Transformer sıralamayı kendiliğinden bilmez; konum bilgisi ve bağlam bunu görünür kılar. Örnek: "Köpek çocuğu izledi." ile "Çocuk köpeği izledi." — aynı kelimeler, farklı sıra, farklı anlam.

Önemli ayırım: Context kalıcı hafıza değildir; modelin o an işleyebildiği çalışma alanıdır.

6.6 Transformer, self-attention ve multi-head attention

- Transformer dil ilişkilerini paralel işler; her tokenı bağlamına göre yeniden temsil eder. Katmanlar bu işlemleri tekrarlayarak daha karmaşık örüntüler öğrenir.
- Self-attention (Query-Key-Value):** her token, bağlamdaki diğer tokenlara bakarak hangi bilginin önemli olduğunu hesaplar. Örnek: "Ayşe kitabı masaya koydu çünkü o ağırdu." — bağlama göre zamirin işaret ettiği bilgi belirlenir.
- Multi-head attention:** farklı attention başlıkları farklı ilişki türlerini izler.
- Causal mask:** geleceğe bakmayı engeller; eğitimde modelin doğru cevabı önceden görerek kopyalamasını önler.

Not: Başarılı bir LLM mimarisi yalnızca attention'dan ibaret değildir.

6.7 Parametreler, model boyutu ve bağlam yönetimi

Parametreler eğitim sırasında öğrenilen sayısal ağırlıklardır; kalite yalnızca parametre sayısı ile açıklanmaz. Sunumun ilkesi: "en büyük model değil, görev için yeterli model."

- Veri kalitesi:** doğru, çeşitli ve görevle ilgili veri davranışı belirler.
- Mimari ve eğitim:** mimari tercihleri ve eğitim yaklaşımı çıktıyı etkiler.
- Görev uyumu:** model gerçek kullanım senaryosunda değerlendirilmelidir.
- Gecikme ve maliyet:** daha büyük model her zaman daha verimli çözüm değildir.

Uzun bağlam; maliyet, gecikme ve önemli bilgiyi kaçırma riskini artırabilir. Çözüm yaklaşımları: bölümlendirme, seçici retrieval, özetleme ve RAG.

Model girdisi: Yanıt yalnızca kullanıcı mesajından değil; bağlama eklenen tüm talimat, veri ve kurallardan şekillenir — bu nedenle güçlü talimat tasarımı bir mimari karardır.

6.8 Pretraining, veri kalitesi ve inference

- Pretraining / next-token prediction: model büyük ölçekli metin örüntülerinden sonraki tokenı tahmin etmeyi öğrenir. Her tahmin hatası, parametrelerin dildeki ilişkileri daha iyi temsil edecek biçimde güncellenmesine rehberlik eder.

- Eğitim verisindeki riskler doğrudan yanıtların güvenilirliğine yansır; deduplication tekrarları azaltarak ezberleme riskini sınırlar.
- Inference: model her adımda olası devamları skorlar; seçilen token bağlama eklenir ve süreç tekrarlanır. Üretim autoregressive ilerler — her yeni token bir sonraki tahminin bağlamına dönüşür.

Risk: Erken seçilen hatalı bir token, sonraki tüm çıktıyı etkileyebilir.

6.9 Halüsinasyon, güvenilirlik ve araç kullanımı

Doğrulama zinciri üç katmandan oluşur:

- RAG:** ilgili kurumsal bilgiyi bağlama ekler.
- Araç ve API:** güncel veri ve hesaplamayı doğrular.
- İnsan denetimi:** yüksek etkili kararları onaylar.

İlke: Akıcılık doğruluk değildir. Model aracı önerir; işlem, yetkili uygulama katmanında güvenlik kontrolleriyle yürütülür.

6.10 RAG ve fine-tuning ayrımı

Boyut	RAG	Fine-tuning
Ne yapar?	İlgili belgeleri bulur, bağlama ekler	Davranışı, üslubu ve görev uyumunu geliştirir
En uygun olduğu durumlar	Güncel bilgi, kurumsal dokümanlar, stok/fiyat/prosedür	Özel terminoloji, tutarlı format, tekrarlayan görevler
Modele etkisi	Model parametreleri değişmez	Model davranışı uyarlanır

Ana mesaj: RAG ve fine-tuning birbirinin alternatifi değil tamamlayıcıdır: RAG doğru ve güncel bağlam sağlar, fine-tuning bu bilgiyle nasıl çalışılacağını daha tutarlı hale getirir.

6.11 Sınırlılıklar ve güvenlik riskleri

Akılcı bir yanıt; güncel, doğru, güvenli veya yetkili olduğu anlamına gelmez. Üç ana risk başlığı:

- Güncellik ve doğruluk sınırı:** yeni haberler, yasalar ve anlık kayıtlar model bilgisinde yer almayabilir — güncel bilgi için API, veri tabanı veya RAG kullanılmalıdır.
- Prompt injection:** güvenilmeyen içerik sistem talimatlarını etkisiz bırakmaya çalışabilir — belge içeriği talimattan ayrı değerlendirilmelidir.
- Gizlilik ve hassas veri:** kişisel, ticari veya erişim bilgileri kontrolsüz işlenmemelidir; erişim, saklama, silme ve eğitimde kullanım politikaları doğrulanmalıdır.

Temel koruma katmanları

- Girdi filtreleme
- Erişim kontrolü
- Çıktı doğrulama
- Kayıt ve izleme

Zorunlu kontrol noktası: Yüksek riskli işlemlerde insan onayı zorunludur.

6.12 Değerlendirme ve model seçimi

Başarı; görev kalitesi, maliyet ve çalışma koşullarının birlikte değerlendirilmesidir.

Kriter	Neyi ölçer
Doğruluk	Kaynak doğruluğu, halüsinasyon oranı, görev sonucu
Gecikme	Yanıt hızı ve kullanıcı deneyimi
Talimat ve format	İstenen ton, yapı ve kurallara uyum
Maliyet	Token, araç ve operasyon yükü

İlke: Benchmark tek başına yeterli değildir; modeli gerçek kullanım senaryolarıyla test edin ve göreve uygun modeli seçin.

7. Modül 5 — Agentic AI ve Yapay Zekâ Ajanları

Alt başlık: "LLM'lerden Otonom Sistemlere Dönüşüm: Mimari, Güvenlik ve Uygulama Rehberi". Sunumun en kapsamlı modülüdür.

7.1 Bir ajan nasıl çalışır? (giriş örneği)

Modül, iki akışı karşılaştırarak başlar. Klasik akış: soru gelir → LLM üretir → cevap. Ajan akışı: soru gelir → kontrol mantığı (reasoning) → veri tabanı, web arama, hafıza ve API gibi araçlar → LLM üretir → cevap.

Örnek senaryo: "Benim bu seneki yıllık izin tatilimi planlayabilir misin?" Ajan; kullanıcı tercihlerini, izin günlerini tutan veri tabanını ve hava durumu tahminlerini kullanır ve gerçek veriye dayalı olarak "Mevcutta izniniz bulunmuyor" gibi doğru bir yanıt üretebilir.

7.2 Ajan kavramı: LLM ile fark

Yapay zekâ ajanı; belirli bir hedef doğrultusunda çevresini gözlemleyen, aksiyonlara karar veren, araçlarla etkileşen ve sonuçlara göre davranışını güncelleyebilen otonom sistemdir. Temel döngü: Gözlem → Karar → Aksiyon → Güncelle.

Boyut	Standart LLM	Agentic sistem
Çıktı niteliği	Olasılıksal cevap	Hedef odaklı davranış
Kapsam	Yalnızca metin	Araç erişimi
Durum	Statik	Kalıcı durum (state)
Etki	İşlem yetkisi yok	Gerçek dünya etkisi

7.3 Temel mimari bileşenleri (7 bileşen)

Bileşen	Rolü
Hedef (Goal)	Somut, ölçülebilir ve başarı kriterleri net tanımlanmış amaç
Model (LLM)	Niyet anlama, karar verme ve araç seçimi yapan çekirdek
Talimatlar	Ajanın rolünü, üslubunu ve operasyonel sınırlarını belirleyen direktifler
Çevre (Environment)	ERP, API ve dosya sistemleri gibi etkileşime girilen dış dünya
Araçlar (Tools)	Gerçek dünyada işlem yapmayı sağlayan fonksiyonel yetenekler ve API'ler
Hafıza (Memory)	Görev bağlamını ve geçmiş etkileşimleri tutan yapılandırılmış katman
Güvenlik & Guardrails	İnsan onayı, yetki kontrolü, maliyet sınırları ve durdurma koşulları

İyi hedef tanımı: Yalnızca ne yapılacağını değil, "onaysız ödeme yapmama" gibi sınırları ve başarı kriterlerini de içermelidir.

7.4 Ajan döngüsü: gözlem, aksiyon ve durdurma

Beş adımlı döngü: Hedefi yorumla → Gözlemle → Planla & seç → Aksiyon al → Doğrula (devam et veya döngüyü sonlandır).

Gözlem kaynakları	Temel aksiyonlar	Durdurma nedenleri
Arama sonuçları (web, doküman, DB)	Arama & okuma	Hedef tamamlandı
Sistem yanıtları (API çıktıları, hata mesajları)	İşlem çağrıları (API, SQL)	Onay gerekli
Bağlam (kullanıcı mesajı, takvim, sensörler)	Hesaplama	Sınır aşımı (adım/maliyet)
Dosya sistemi (belgeler, loglar)	Kod çalıştırma	Güvenlik / politika ihlali

Kritik uyarı: Durdurma koşulu olmayan ajan sonsuz döngü riski taşır.

7.5 Otonomi seviyeleri ve risk yönetimi

Seviye	Ad	Davranış
Seviye 0	Sıfır otonomi	Yalnızca metin tabanlı cevap üretimi; dış dünya etkileşimi yok
Seviye 1	Öneri modu	Sonraki aksiyonu belirler ve önerir; karar ve uygulama insandadır
Seviye 2	Onaylı aksiyon	Plan hazırlar ve aksiyon seçer; her adımda insan onayı şarttır
Seviye 3	Sınırlandırılmış	Düşük riskli ve tanımlı alanlarda otonom işler; kritik eşiklerde durur
Seviye 4	Geniş otonomi	Uzun süreli, çok adımlı ve karmaşık hedeflerde geniş yetkiyle bağımsız çalışır

Risk ve kontrol dengesi

- Yüksek riskli görevler** (finansal işlemler ve ödeme onayları, sözleşme imzalama veya fesih, çalışan hesabı kapatma/yetki verme): Seviye 2 — human-in-the-loop zorunludur.
- Düşük riskli görevler** (web arama ve belge özetleme, takvim planlama ve e-posta taslağı, grafik oluşturma ve rapor hazırlama): Seviye 3 veya 4 otonomi uygulanabilir.

7.6 Modelin rolü ve araç kullanımı

Modelin stratejik kararları

- Niyet analizi: kullanıcı hedefini ve karmaşık talepleri anlama.
- Adım belirleme: bir sonraki mantıklı aksiyonu değerlendirme.
- Araç seçimi: görev için en uygun dış fonksiyonu belirleme.
- Parametre üretimi: araç için gerekli yapılandırılmış veriyi hazırlama.
- Onay ihtiyacı: insan müdahalesi gerekip gerekmediğine karar verme.

Kritik sınır: Yetki, ödeme limitleri ve güvenlik politikaları gibi iş kuralları sadece modele bırakılmamalıdır.

Araç ekosistemi

- Güncel bilgi: web araması ve gerçek zamanlı haber akışları.
- Kesin hesaplama: calculator veya kod yürütme.
- Kurumsal kaynaklar: veri tabanı sorguları, API entegrasyonları, ERP erişimi.
- Gerçek işlemler: e-posta gönderimi, dosya yönetimi, takvim planlama.
- Yetkili işlem: kurum içi sistemlerde değişiklik yapma yetkisi.

Teknik köprü: function calling ve tool schema

Araç şeması dört parçadan oluşur: tanımlayıcı (isim ve açıklama), girdiler (parametreler ve tipler), kısıtlar (zorunlu alanlar/değerler) ve çıktı (sonuç ve hata biçimi). Model bu yapılandırılmış şemayı kullanarak dış dünyaya "ne sorması gerektiğini" ve "hangi formatta cevap beklediğini" öğrenir.

7.7 Araç tasarımı, doğrulama ve hata yönetimi

- **Read araçları:** sadece bilgi alır, sistem durumunu değiştirmez — düşük risklidir.
- **Write araçları:** veri veya işlem değiştirir — kuvvetli onay ve denetim gerektirir.
- **Kapsam tasarımı:** aşırı güçlü araçlar yerine dar kapsamlı, spesifik araçlar tercih edilmelidir.
- **Idempotency:** aynı çağrı tekrarlandığında ikinci bir yan etki oluşmasını engeller; benzersiz işlem anahtarı kullanılır.
- **Sonuç doğrulama:** araç yanıtı şema, yetki, tarih ve mantıksal sınırlar açısından kontrol edilmelidir.

Tipik hatalar: zaman aşımı, yetersiz yetki, geçersiz parametre, servis yoğunluğu, çelişkili sonuç. Tepki stratejileri: yeniden dene, alternatif araç, insan eskalasyonu.

Kritik uyarı: Aynı parametreyle kör tekrar yapılmamalıdır; hata durumunda güvenli durma veya kullanıcıdan bilgi alma tercih edilmelidir.

7.8 Planlama yaklaşımları ve state yönetimi

- **Sabit plan:** süreç iyi biliniyorsa deterministik bir workflow olarak kodlanır.
- **Dinamik plan:** değişen bilgi ve çevresel gözlemlere göre anlık güncellenir.
- **Planla-Uygula:** önce tam plan oluşturulur, ardından adımlar yürütülür.
- **ReAct (Reason + Act):** değerlendirme ve aksiyonun çevre gözlemleriyle dönüşümlü yürütülmesi; bilgi eksikliği, bütçe aşımı veya güvenlik kuralı durumunda ajana yeniden yön verir.

State (durum) kavramı

State, ajanın görevle ilgili güncel ve yapılandırılmış bilgisidir. Konuşma geçmişinden farklı olarak makine tarafından işlenebilir bir görev özeti sunar. Bileşenleri:

Alan	İçerik
Ana hedef	Somut ve ölçülebilir amaç
Kısıtlar	Bütçe, tarih, yetki
İlerleme	Tamamlanan adımlar
Onay durumu	Bekleyen izinler
Hata logu	Geçersiz parametreler ve hatalar
Seçenekler	Belirlenen alternatifler

Neden önemli? State yönetimi, ajanın uzun süreli görevlerde bağlamı kaybetmemesini ve hata durumunda güvenli noktadan devam etmesini sağlar.

7.9 Hafıza, reflection ve bağımsız doğrulama

Ajan hafıza katmanları

Ajanlar yalnızca anlık bağlamı değil, görev boyunca biriktirdikleri bilgileri farklı bellek türlerinde saklar: çalışma belleği, episodik hafıza, semantik hafıza ve prosedürel hafıza. Bağlam geçicidir; hafıza seçici ve kalıcıdır.

Reflection (öz-değerlendirme)

- Ajanın kendi geçmiş adımlarını, araç çıktılarını ve plan uyumluluğunu analiz etmesi; hataları tespit edip döngü içinde dinamik davranış düzeltmesi yapması.
- **Kritik ayırım:** reflection, modelin yeniden eğitilmesi (retraining) değildir; mevcut görev döngüsünde akıl yürütme stratejisinin güncellenmesidir.
- **Fayda:** halüsinasyon riskini azaltır, sonsuz döngüleri engeller ve karmaşık hedeflerde başarı oranını artırır.

Bağımsız doğrulama mekanizmaları

- JSON şema kontrolü
- Birim testleri ve kod testi
- Deterministik iş kuralları
- İkinci model ile çapraz kontrol (cross-check)

7.10 İnsan onayı ve güvenlik protokolleri

Human-in-the-Loop (HITL)

- **Kritik tetikleyiciler:** finansal işlemler, veri silme, yetki değişiklikleri ve hukuki taahhütler mutlaka onay gerektirir.
- **Onay yorgunluğu:** her küçük işlem için onay istemek yerine risk ve maliyet bazlı eşikler (guardrails) tanımlanmalıdır.
- **Geri alınabilirlik:** işlemler taslak, ön izleme veya test ortamı gibi geri alınabilir biçimde tasarlanmalıdır.

İdeal onay ekranı; yapılacak işlemi, gerekçesini, hedef sistemi, potansiyel riskleri ve varsa güvenli alternatifleri kullanıcıya sunar.

Güvenlik ve guardrails

- **Minimum yetki ilkesi:** ajanlara yalnızca görevlerini tamamlamak için gereken en dar kapsamlı erişim verilmelidir.
- **Sandbox izolasyonu:** kod çalıştırma ve dosya erişimi ana sistemden izole güvenli alanlarda sınırlandırılmalıdır.
- **Döngü kontrolü:** maksimum adım sayısı ve bütçe sınırlarıyla sonsuz döngü engellenmelidir.

Guardrail çeşitleri: girdi/çıkış filtreleme, araç parametre kontrolü, maliyet sınırı, veri gizliliği maskeleyme, içerik güvenliği.

Temel soru: Ajan ne kadar bağımsız olabilir değil; hangi kararlar kural ve insan kontrolüne bırakılmalıdır?

7.11 Prompt injection ve sistem dayanıklılığı

- **Dolaylı (indirect) injection:** e-posta, dosya veya veri tabanı içeriğindeki kötü niyetli metinlerin ajan talimatı olarak algılanması.
- **Aksiyon dönüşümü:** sıradan bir metnin ajan tarafından yetkili araç kullanımına (veri silme, e-posta gönderme) dönüştürülmesi.
- **Güven hiyerarşisi:** sistem talimatı > kullanıcı talebi > dış kaynaklar. Dış kaynak içerikleri asla doğrudan "görev talimatı" olarak kabul edilmemelidir.

Döngü önleme ve denetim

- Maksimum adım sayısı, maliyet sınırı ve tekrar eden araç çağrılarının otomatik tespiti.
- Ara doğrulamalar: uzun görevlerde kritik aşamalarda plan ve verilerin geçerliliğinin kontrolü.

- **Checkpoint:** plan, tamamlanan adımlar, araç sonuçları ve maliyet bilgisinin kalıcı saklanması.
- **Resume:** hata veya kesinti sonrası ajanın en son güvenli noktadan devam edebilmesi.

7.12 Tek ajan ve çok ajanlı orkestrasyon

Boyut	Tek ajan (Single Agent)	Çok ajanlı (Multi-Agent)
Ana avantaj	Basitlik ve verimlilik; plan yapar, araç seçer, yanıtı tek başına üretir	Uzmanlaşma ve ölçek; görevler uzman rollere dağıtılır
Maliyet	Daha düşük maliyet, daha az mesaj trafiği	Karmaşık mesaj trafiği ve artan API maliyeti
Doğruluk	Koordinasyon hatası riski minimum, net kontrol	Her ajan daha az araçla daha yüksek doğrulukta çalışır
Risk	Uzun talimatlarda rol çakışması ve bağlam kaybı	Orkestrasyon karmaşıklığının yönetilmesi gerekir

Orkestrasyon desenleri

- **Supervisor modeli:** merkezî bir yönetici ajan hedefi analiz eder, alt görevleri belirler ve uzman ajanlara dağıtarak süreci orkestre eder.
- **Handoff (görev devri):** bir aşama tamamlandığında kontrolün ve mevcut state'in sonraki uzman ajana aktarılması.
- **Debate ve eleştiri:** ajanların aynı problemi farklı açılardan inceleyerek birbirlerinin çıktılarını doğrulaması ve iyileştirmesi.

7.13 Ajan frameworkleri ve seçim kriteri

- Orkestrasyon ve döngü: agent loop, tool calling ve çoklu ajan yönetimi.
- Durum ve hafıza: checkpoint, resume ve uzun süreli görev takibi altyapısı.
- Gözlemlenebilirlik: tracing, guardrail entegrasyonu ve insan onayı mekanizmaları.
- Uzmanlaşmış handoff: görevlerin uygun alt ajanlara devrinin otomatikleştirilmesi.

Seçim kriteri: Basit ajanlar için doğrudan API kullanımı yeterlidir; karmaşık state, çoklu ajan etkileşimi, onay süreçleri ve dayanıklılık gereken projelerde framework kullanımı zorunluluktur.

7.14 Tracing, değerlendirme ve test

Uçtan uca izleme

- Tracing vs logging: trace, tekil olayların ötesinde kullanıcı görevini uçtan uca takip eder.
- Kayıt bileşenleri: niyet, plan adımları, araç çağrıları, parametreler, onaylar ve hatalar.
- Analiz ihtiyacı: hata kaynağını bulma ve modelin karar mekanizmalarını denetleme.

Gizlilik: Trace kayıtları maskeleyme, şifreleme ve PII temizleme süreçlerinden geçmelidir.

Başarı metrikleri

- Görev performansı: nihai başarı oranı, plan kalitesi, araç seçim doğruluğu.
- Operasyonel verim: maliyet, gecikme (latency), döngü/tekrar oranları.
- Güvenlik denetimi: yetki ihlalleri, write başarı oranı, durdurma kararları.
- Sunumdaki örnek eşikler: %98 araç doğruluğu, 2 saniyenin altında ortalama gecikme.

Test ve validasyon

- Test seti içeriği: başlangıç state'i, araçlar, yasaklı aksiyonlar ve başarı koşulları.
- Kritik senaryolar: eksik bilgi, yetki yetersizliği, sonsuz döngü tespiti.
- Riskli işlemler: geri alınamaz aksiyonların ve insan onay mekanizmalarının testi.
- Bağımsız doğrulayıcılar (rule-based, JSON Schema, code test) ile ek güven katmanı.

7.15 Ne zaman ajan kullanılır? Uygulama alanları

Alan	Ajanın yaptığı iş	Dikkat edilecek risk
Müşteri hizmetleri	Niyet anlama, kimlik doğrulama, dinamik çözüm, gerektiğinde insana eskalasyon	Niyet analizi ve otomasyon dengesi
Veri analizi	Doğal dilden SQL üretimi, veri tabanı sorgulama, grafiksel özetleme	Yanlış veya pahalı sorgu
Yazılım geliştirme	Kod deposu inceleme, değişiklik planlama, test yazımı, Pull Request oluşturma	Sandbox ve branch sınırı
Araştırma ajanları	Web ve iç kaynaklarda derin araştırma, kaynak değerlendirme, sentez	Kaynak halüsinasyonu
İnsan kaynakları	Kurum politikası sorgulama, izin yönetimi	Hassas veride insan onayına yönlendirme

Uygunluk kriterleri (ajan gerekli mi?)

- Görev çok adımlı ve dinamik mi?
- Modelin karar vermesi gerekiyor mu?
- Dış araç kullanımı zorunlu mu?
- Başarı somut olarak ölçülebilir mi?
- İşlem geri alınabilir mi?

7.16 Modül kapanışı ve temel mesajlar

Güvenilir bir Agentic AI sistemi, otonom yetenekler ile insan denetimi arasındaki hassas dengenin sonucudur. LLM'ler yanıt üretirken ajanlar hedef odaklı aksiyon alır; başarıyı belirleyen, modelin zekâsı kadar sistemin güvenlik ve kontrol mimarisidir.

- **Stratejik otonomi:** risk seviyesine göre kademelendirilmiş yetki, kritik işlemlerde zorunlu insan onayı, minimum yetki ilkesi.
- **Operasyonel şeffaflık:** uçtan uca tracing, somut başarı metrikleri, checkpoint mekanizmaları, sürekli plan doğrulama döngüleri.
- **Teknik dayanıklılık:** idempotency, guardrail sistemleri, prompt injection koruması, izole sandbox ortamları.

8. Modül 6 — MCP (Model Context Protocol)

Verinin Mutfağı kimliğiyle hazırlanmış 17 slaytlık teknik alt modül. Künye: "MCP Nedir, Nasıl Kullanılır? — Model Context Protocol ile Yapay Zekâ Uygulamalarını Gerçek Dünyaya Bağlamak", referans MCP 2026-07-28.

8.1 Problem: herkes kendi adaptörünü yazıyordu

MCP öncesinde her yapay zekâ uygulaması, her veri kaynağına bağlanmak için sıfırdan özel entegrasyon yazmak zorundaydı. Postgres, Slack, GitHub, Google Drive, Jira ve S3 gibi altı kaynak birden fazla AI uygulamasıyla eşleştiğinde entegrasyon sayısı $M \times N$ ölçeğinde katlanarak artıyordu.

- Her AI uygulaması, her veri kaynağı için ayrı özel entegrasyon kodu yazıyordu.
- Aynı bağlantı işi onlarca ekipte birbirinden habersiz tekrarlanıyordu.
- Bir uygulama için yazılan entegrasyon, başka bir uygulamada yeniden kullanılamıyordu.

Ölçek karşılaştırması: 3 uygulama $\times$ 6 kaynak = 18 bağlantı yerine $3 + 6 = 9$ bağlantı. MCP, entegrasyon problemini çarpma işleminden toplama işlemine çevirir ($M \times N \rightarrow M + N$).

8.2 Tek cümlede MCP

MCP, yapay zekâ uygulamalarının dış dünyadaki araçlara, verilere ve sistemlere standart bir şekilde bağlanmasını sağlayan açık bir protokoldür.

Özellik	Anlamı
Açık standart	Linux Foundation çatısı altında yürütülür; tek bir şirkete ait değildir
Model bağımsız	Claude, GPT veya Gemini gibi farklı modellerle kullanılabilir
İki yönlü	Yalnızca veri okumayı değil, izin verilen aksiyonları almayı da destekler
Taşınabilir	Bir kez yazılan bir server, uyumlu farklı client uygulamalarında çalışır

Analoji: MCP, yapay zekâ uygulamaları için USB-C gibi ortak bir bağlantı standardıdır; her cihaz için ayrı kablo ihtiyacını azaltır.

8.3 Bir MCP sunucusu ne sunar? (mutfak metaforu)

"Tools mutfak aletleri, Resources malzemeler, Prompts tariflerdir." Model şeftir; MCP ise bunların standart bağlantı fişidir.

Primitive	Tanım	Örnek
Tools	Modelin çalıştırdığı fonksiyonlar; aksiyon alır, yan etki üretebilir	sorgu_calistir · ticket_ac · rapor_gonder
Resources	Modelin okuduğu salt okunur bağlam: şema, doküman, log	tablo_şeması · dbt model · runbook
Prompts	Kullanıcının seçtiği hazır iş akışı şablonları	"Bu tabloyu profile" · "PR'ı gözden geçir"

Sunumda ayrıca MCP Apps, Tasks ve Enterprise Managed Auth başlıkları ekosistemin yeni bileşenleri olarak anılır.

8.4 Protokolün altındaki mekanik

- **JSON-RPC 2.0:** tüm MCP mesajları JSON-RPC 2.0 biçimindedir (örnek: `{"jsonrpc":"2.0","method":"tools/call","params":{"name":"sorgu_calistir"},"id":1}`). Her istek kendini tanımlar; protokol sürümü ve istemci bilgisi istekle birlikte taşınır.
- **stdio transport:** server yerelde alt süreç olarak çalışır; masaüstü ve CLI senaryoları için uygundur.
- **Streamable HTTP transport:** uzak server bağlantıları için; sıradan HTTP yükü gibi işletilebilir.

28 Temmuz 2026 sürümüyle öne çıkan mekanikler: stateless çekirdek, Mcp-Method ve Mcp-Name başlıkları, cache'lenebilir liste yanıtları ve kullanıcı kapsam belgesi.

8.5 Uçtan uca örnek: "Geçen ayın cirosu ne kadar?"

Adım	Ne olur
1. Soru	Kullanıcı doğal dille sorgusunu Host'a iletir
2. Keşif	Client, bağlı Server'lardan cache'lenebilir araç listesini alır
3. Seçim	Model uygun aracı seçer ve parametreleri üretir
4. Onay	Host, kritik aksiyon için kullanıcıdan açık onay ister
5. İcra	Server aracı çalıştırır, veriye erişir ve sonucu döndürür
6. Yanıt	Model sonucu doğal dile çevirerek kullanıcıya sunar

İpucu: Eksik bilgi durumunda Server bilgi talebi döner; süreç MRTR kalıbıyla tekrarlanır.

8.6 Ajan yığınınında MCP nerede duruyor?

Katman	Sorumluluğu
Model	Muhakeme, dil ve karar üretir — "Ne yapılmalı?"
Ajan katmanı	Planlama, hafıza, döngü kontrolü, hata yönetimi ve bütçe (Claude Agent SDK, LangGraph, CrewAI veya özel kod)
MCP	Araç ve bağlam katmanı — "Neye erişebiliyorum?"
Dış sistemler	Veritabanları, SaaS API'leri, dosya sistemleri ve dahili servisler

MCP'nin olmadığı şeyler

- Ajan framework'ü değildir; planlama, hafıza, muhakeme ve döngü kontrolünü yönetmez.
- Function calling'in yerine geçmez; araçların nereden geldiğini ve nasıl sunulduğunu standartlaştırır.
- Ajanlar arası iletişim protokolü değildir; temel ekseni agent ↔ araç ilişkisidir.
- Tek bir modele veya sağlayıcıya bağlı değildir.

Ana mesaj: Ajan mimarisinde MCP, "ne yapacağım" katmanı değil, "neye erişebiliyorum" katmanıdır.

8.7 Döngünün içinde MCP ve 2026-07-28 etkileri

Temel ajan döngüsü: Hedef → Planla → MCP ile araç çağır → Gözlemle → Değerlendir → yeni tur veya sonuç. Reasoning iki noktada devrededir: planlama (hangi araç, hangi sırayla, hangi parametrelerle) ve değerlendirme (sonuç beklentiye uygun mu; devam, düzeltme veya durma). ReAct kalıbında MCP eylem ve gözlem adımlarını standartlaştırır; düşünme model ve ajan katmanının sorumluluğudur.

Yenilik	Ajanlara etkisi
Tasks	Saatler süren işleri tasks/get ile yoklama, tasks/update ile müdahale
MRTR / elicitation	Açık bağlantıya bağlı kalmadan kullanıcıdan bilgi veya onay isteme
Stateless çekirdek	Çok sayıda eşzamanlı ajan koşusunu standart altyapıda yönetme
Cache'lenebilir araç listeleri + subscriptions/listen	Prompt cache sürekliliği, maliyet ve gecikme azalması

Kritik tasarım ilkesi: Tool adı ve açıklaması, modelin seçim muhakemesine giren temel ipuçtur. Zayıf açıklama yanlış araç seçimine yol açar; çok geniş kataloglar bağlam maliyetini ve seçim hatasını artırır. Çözüm: az ve keskin araç setleri + gateway seviyesinde filtreleme. Progressive discovery, katalogu konuşma daraldıkça kademeli açar.

8.8 MCP nerede kullanılıyor? (benimseme)

Gösterge	Değer	Kaynak
TypeScript ve Python SDK toplam indirme	1 milyar+	MCP Blog, Temmuz 2026
Tier 1 SDK'ların aylık indirme hacmi	~500 milyon / ay	MCP Blog, Temmuz 2026
Honeycomb'da ajanlar tarafından yapılan aylık etkileşimli sorgu oranı	%20	MCP Blog, Temmuz 2026

- **Ekosistemde öne çıkan platformlar:** GitHub, Slack, Sentry, Linear, Figma, Supabase, Postgres, Google Drive, Notion.
- **Bulut altyapısında MCP server barındırma:** Amazon Bedrock AgentCore (ajan altyapısıyla entegre), Microsoft Foundry (kurumsal ölçekte yönetim), Google Cloud (dağıtım senaryoları), Cloudflare Workers (edge'de hafif ve ölçeklenebilir).

8.9 Nereden başlamalı?

Sonraki adım: Bu hafta bir tane bağla, bir tane yaz. Standart bağlantı hazır; şimdi kendi veri mutfağındaki ilk aracı güvenli biçimde bağlama zamanı.

- $M \times N \rightarrow M + N$: açık standart, entegrasyonu çarpmadan toplamaya çevirir.
- `host · client · server`: model veriye doğrudan dokunmaz; zincir güvenli erişimi taşır.
- Stateless çekirdek: 2026 sonrası yeni mimari; ölçeklenebilir ve yönetilebilir.
- Başvuru kaynağı: modelcontextprotocol.io

10. Sunumun Ana Çıkarımları

- **1. Yapay zekâ birikimli bir evrimdir.** 1943'ten bugüne uzanan seksen yıllık bir bilimsel birikimin ürünüdür; ChatGPT bir icat değil, bu birikimin erişilebilir arayüzüdür.
- **2. Veri, modelin üst sınırıdır.** Garbage in, garbage out: model öğrendiği verinin doğruluğu ve temsil gücü kadar güvenilirdir.
- **3. Akıcılık doğruluk değildir.** Halüsinasyon bir arıza değil, olasılıksal üretimin doğal sonucudur; doğrulama zinciri (RAG + araç + insan) zorunludur.
- **4. Ajan = LLM + hedef + araç + hafıza + guardrail.** LLM yanıt üretir; ajan hedef odaklı aksiyon alır ve gerçek dünyada etki yaratır.
- **5. Otonomi bir düğme değil, bir skaladır.** Seviye 0'dan 4'e kademelendirilmiş yetki, risk düzeyine göre insan onayı ile dengelenmelidir.
- **6. Durdurma koşulu olmayan ajan risktir.** Adım ve maliyet sınırları, checkpoint/resume ve idempotency teknik dayanıklılığın temelidir.
- **7. MCP erişim katmanıdır, akıl katmanı değildir.** "Ne yapacağım" sorusunu ajan katmanı, "neye erişebiliyorum" sorusunu MCP yanıtlar.
- **8. Standartlaşma maliyeti düşürür.** $M \times N$ entegrasyon yükünü $M + N$ 'e indiren açık protokol, ekosistemi taşınabilir hale getirir.
- **9. Başarı bütünseldir.** Teknoloji + veri + süreç + insan = sürdürülebilir etki. Demo başarısı ürün başarısını garanti etmez.
- **10. Temel soru değişti.** "Ajan ne kadar bağımsız olabilir?" değil, "hangi kararlar denetime bırakılmalıdır?"

Kaynakça Link:

https://docs.google.com/document/d/1fbiP9yBx_rZVbhIVWRy71sO3OotZt-7WxExwsY8BYmE/edit?tab=t.0

<https://docs.google.com/spreadsheets/d/1TIYNnJlcBqUe2aBxSBtfj7Cf5la9JxDP9U1RKR13NYg/edit?gid=0#gid=0>

Verinin Mutfağı · Ferhat İşyapan · VM Yazılım Eğitim ve Danışmanlık A.Ş.